Alexandre Teixeira

alex.dinis.06@gmail.com | github.com/AlexDTT | linkedin.com/in/alexandre-teixeira-186093278

Education

Colégio Internato dos Carvalhos | *GPA: 20.0/20* — *Sep 2021 - Jun 2024*
High School Diploma | *Informática* — *Carvalhos, Vila Nova de Gaia*

Faculty of Engineering, University of Porto — *Sep 2024 - Jun 2027*
B.S. | *Informatics and Computing Engineering* — *Porto, Portugal*

Experience

Software Engineering Intern — *Jul 2026 - Aug 2026*
INESC TEC — *Porto, Portugal / Vila Real, Portugal*

- Built a full-stack prototype for AI-assisted football video analysis, including chatbot, automated reports and data visualizations (player comparisons, performance dashboards).
- Integrated computer vision for player/ball tracking and semi-automatic match event tagging.
- Collaborated with researchers in an agile, iterative environment.

Full-stack Developer — *Sep 2025 - Mar 2026*
NovaSupplier — *Remote*

- Contributed across a Next.js frontend and Nest.js API for a B2B sourcing platform.
- Worked on features spanning supplier discovery, quote management, orders, and messaging.

Full-stack Intern — *Jul 2023 - May 2024*
Fullscreen — *Serzedo, Vila Nova de Gaia*

- Built BackOffice and FrontOffice features for FirstPharma, a pharmacy management platform - permission systems, user management, and a role-based document repository in Symfony.
- Implemented two-factor authentication (TOTP, Google Authenticator, email) and product attribute management for StudioCMS, Fullscreen's proprietary CMS.
- Integrated a file repository module into the Livraria Lello web application, with custom UI and document-type-specific visuals.

Projects

The AI Scout — *Jul 2026 - Aug 2026*
INESC TEC / FEUP research project

A full-stack prototype exploring how AI can support football analysts and scouts, rather than replace their judgment.

Built during a summer research internship, in a pair with one other developer under academic and industry supervision. My part covered the backend, an AI-assisted chatbot and automated report generator, data visualization (player comparisons, performance dashboards), and integrating a computer vision pipeline for video-based player and ball tracking. I also built a manual and semi-automatic event-tagging system for match analysis, driven by positions the vision pipeline recovered from footage. The brief throughout was to design around analysts' real, often-invisible workflows - not to lead with the AI.

Stack: Computer Vision, AI Chatbot, Full-stack, Data Visualization.

NovaSupplier — *Sep 2025 - Mar 2026*
B2B sourcing platform

A full-stack platform connecting global brands with verified European suppliers, replacing trade fairs and manual coordination with a digital-first workflow.

The platform covers the full supply chain loop in one place - supplier discovery, quote management, orders, invoices, shipping, and messaging. I contributed across the stack, working with a Next.js frontend and a Nest.js API backed by PostgreSQL.

Stack: Next.js, Nest.js, Prisma, PostgreSQL, Tailwind CSS, REST API.

Cheetah *Sep 2023 - May 2024*

SaaS scheduling platform

An enterprise client-scheduling platform with intelligent calendar management and real-time synchronization.

Built during my internship at Fullscreen, alongside other work on the company's CMS and client projects. Handled the scheduling logic and calendar sync on top of a Symfony backend.

Stack: Symfony, PHP, MySQL, JavaScript, Twig.

Boavista Kickboxing *May 2023 - Jun 2023*

Gym management platform

A gym management system covering member registration, class scheduling, attendance, and a merchandise store.

One of my first projects, made with some other colleagues in highschool. The project included member registration, class scheduling, attendance tracking, weight and performance monitoring, and a small storefront for merchandise, built end to end.

Stack: PHP, MySQL, JavaScript, HTML/CSS.

University Projects

Compiler Register Allocation *May 2026 - May 2026*

3-person team · Algorithm Design

- Built a global register allocator that merges live ranges into webs, constructs an interference graph, and colors it under a bounded register count
- Implemented baseline coloring, bounded spilling, live-range splitting, DSatur ordering, graph-class fast paths, and guarded branch-and-bound refinement
- Added a custom free-split recovery mode, marker-preserving output, DOT graph exports, deterministic integration tests, and presentation-ready visualizations

This C++ compiler back-end tool takes variable live ranges, merges compatible ranges into webs, builds an interference graph, and allocates those webs to a limited set of physical registers.

Beyond the supplied greedy baseline, our custom allocator combines graph-class shortcuts, DSatur ordering, and a bounded branch-and-bound pass for smaller graphs. When coloring cannot avoid memory, the tool supports explicit spilling and marker-preserving live-range splitting, including a free_split recovery strategy that evaluates candidate splits and keeps only measurable improvements.

ARC Gym *Mar 2026 - Jun 2026*

3-person team · Web Languages and Technologies

- Co-built a framework-free gym management platform in strict PHP, SQLite, semantic HTML, layered CSS, and vanilla JavaScript/AJAX
- Implemented class enrollment and waitlists, trainer/member/admin workflows, personal training, equipment reservations, memberships, notifications, and analytics
- Added QR membership check-in, physical cards, a Bearer-authenticated JSON/XML REST API, migrations, audit logs, backups, session controls, and security hardening

ARC Gym is a full gym-management platform built without application or CSS frameworks: strict PHP, SQLite through PDO, semantic HTML, layered CSS, and vanilla JavaScript/AJAX.

Members can manage memberships, browse and join classes, enter waitlists, reserve equipment, book personal training, review sessions, receive notifications, and present a QR gym pass. Trainers manage availability, rosters, profiles, and analytics; administrators manage the full catalog, users, equipment, promotions, landing content, audit logs, backups, API clients, and check-ins.

The security layer includes prepared statements, output escaping, CSRF tokens, password hashing, session rotation, role guards, upload validation, hashed API tokens, and rate limits. A public REST API supports JSON and XML, while a numbered SQL/PHP migration system keeps schema and seed data reproducible.

Scientific Review Assignment

Mar 2026 - May 2026

3-person team · Algorithm Design

- Modeled reviewer-to-paper assignment as a capacitated bipartite flow network with expertise, review-count, and reviewer-capacity constraints
- Implemented Ford-Fulkerson and Edmonds-Karp in C++, plus batch/interactive workflows, CSV parsing, assignment exports, and reviewer-absence risk analysis
- Documented graph construction, complexity, and algorithm tradeoffs with generated Doxygen and Graphviz visualizations

This C++ tool turns scientific-paper review assignment into a maximum-flow problem. Submissions and reviewers form the two sides of a bipartite network; capacities encode minimum reviews per paper, maximum workload per reviewer, and primary/secondary expertise rules.

Both Ford-Fulkerson and Edmonds-Karp are available so their behavior and complexity can be compared on the same datasets. The application supports an interactive terminal UI and deterministic batch mode, writes assignments to CSV, and can test how removing a reviewer affects coverage.

Clutch

Feb 2026 - Jun 2026

3-person team · Software Engineering

- Co-built a campus peer-help app that matches nearby FEUP students by subject, coordinates live help sessions, and rewards helpers with a karma economy
- Shipped SOS requests, indoor campus maps, real-time offers and chat, QR session verification, reviews, badges, streaks, leaderboards, moderation, and offline-aware flows
- Worked across Expo/React Native, TypeScript, Convex, CI/CD, unit/integration tests, automated Android E2E recordings, release management, and project documentation

Clutch connects FEUP students who are stuck with nearby peers who can help in person. A student chooses a course unit and broadcasts an SOS; available helpers can respond, meet on campus, verify the session by QR code, chat, and review one another afterward.

We built the mobile app with Expo, React Native, TypeScript, and Convex. The product also includes a karma economy, badges and streaks, course/global leaderboards, user blocking, quiet hours, multiple FEUP buildings, capacity information, offline behavior, and a separate privacy-conscious admin analytics dashboard.

Watch walkthrough

Ninjix

Feb 2026 - Jun 2026

4-person team · LCOM

- Co-built a pixel-art tower-defense game and small game engine in C for MINIX, driven by timer, keyboard, mouse, graphics, and serial-port interrupts
- Implemented a 60 Hz simulation/render loop with levels, waves, tower upgrades, enemy abilities, projectile pools, menus, profiling, and custom PNG-to-XPM tooling
- Added two-player UART multiplayer with attacker/defender roles, a binary synchronization protocol, handshakes, retransmission, and shared game-state events

Ninjix was our final Computer Laboratory project: a complete tower-defense game written in C for MINIX 3, without a game framework. The codebase acts as a small event-driven engine layered over the course's hardware work.

The game normalizes hardware interrupts into app events, runs gameplay at 60 Hz, renders through a back buffer, and manages towers, eight enemy types, projectiles, waves, upgrades, targeting modes, menus, overlays, and reusable sprite resources. We also built developer tooling for PNG-to-XPM conversion, Doxygen/Graphviz documentation, and section-level profiling with FlameGraph output.

The multiplayer mode links two machines over COM1 at 115200 bps. One player attacks by spending elixir to spawn enemies; the other defends by placing and upgrading towers. A custom binary protocol handles discovery, role negotiation, gameplay events, pause synchronization, and restart decisions.

Watch walkthrough

TankTussle *Sep 2025 - Jan 2026*

3-person team · Software Design and Testing

- Co-built a TankTrouble-inspired 2D local tank combat game in Java with Lanterna, including local 1v1 multiplayer and single-player AI
- Implemented bouncing projectile physics, ricochets, destructible and indestructible walls, multiple maps, a map editor, menus, settings, sound, music, and particles
- Structured the game around MVC, state and event-driven architecture, design patterns, Gradle automation, JUnit/jqwik/Mockito tests, JaCoCo, Pitest, and Spotless

TankTussle is a TankTrouble-inspired local tank combat game built with Java and Lanterna for LDTS 2025/26.

The game supports local 1v1 multiplayer and single-player matches against AI opponents with three difficulty levels. Combat centers on bouncing projectiles, ricochets, destructible and indestructible walls, multiple arena layouts, particle effects, sound, music, pause and victory flows, customizable controls, and a built-in map editor.

Internally, the project uses MVC, explicit game states, an event-driven loop, and patterns such as factory method, observer, singleton, facade, state, and template method. The Gradle toolchain includes JUnit, jqwik, Mockito, JaCoCo, Pitest, and Spotless for testing, mutation analysis, coverage, and formatting.

Skills

Languages: Python, JavaScript, TypeScript, PHP, C/C++, C#, Java, SQL
Frontend: React, Tailwind CSS, HTML/CSS
Backend: Node.js, Nest.js, Symfony, REST API
Security & Data: Authentication, TOTP/2FA, RBAC, Computer Vision, Data Visualization
Databases & Tools: PostgreSQL, MySQL, Prisma, Git, Linux, Figma, VS Code, Agile
Languages Spoken: Portuguese (Native), English (Advanced)